

Monday

LECTURE - 25

4/5/26

Schedules.

⇒ Serial: No interleaving; T_1 & T_2 runs independently one after another.

• If 3 Transactions; then 3! possibility = $\boxed{6}$. $\begin{matrix} T_1 & T_2 & T_3 \\ T_3 & T_2 & T_1 \\ \vdots & & \end{matrix}$

⇒ Non-Serializable: Parallel schedules w conflict.

⇒ Serializable: Parallel schedules working in a way that they don't cause any conflict & gives a vibe of serial scheduling.

Conflicting Operations

- $r_1(X) \rightarrow w_2(X)$
- $w_1(X) \rightarrow r_2(X)$
- $w_1(X) \rightarrow w_2(X)$

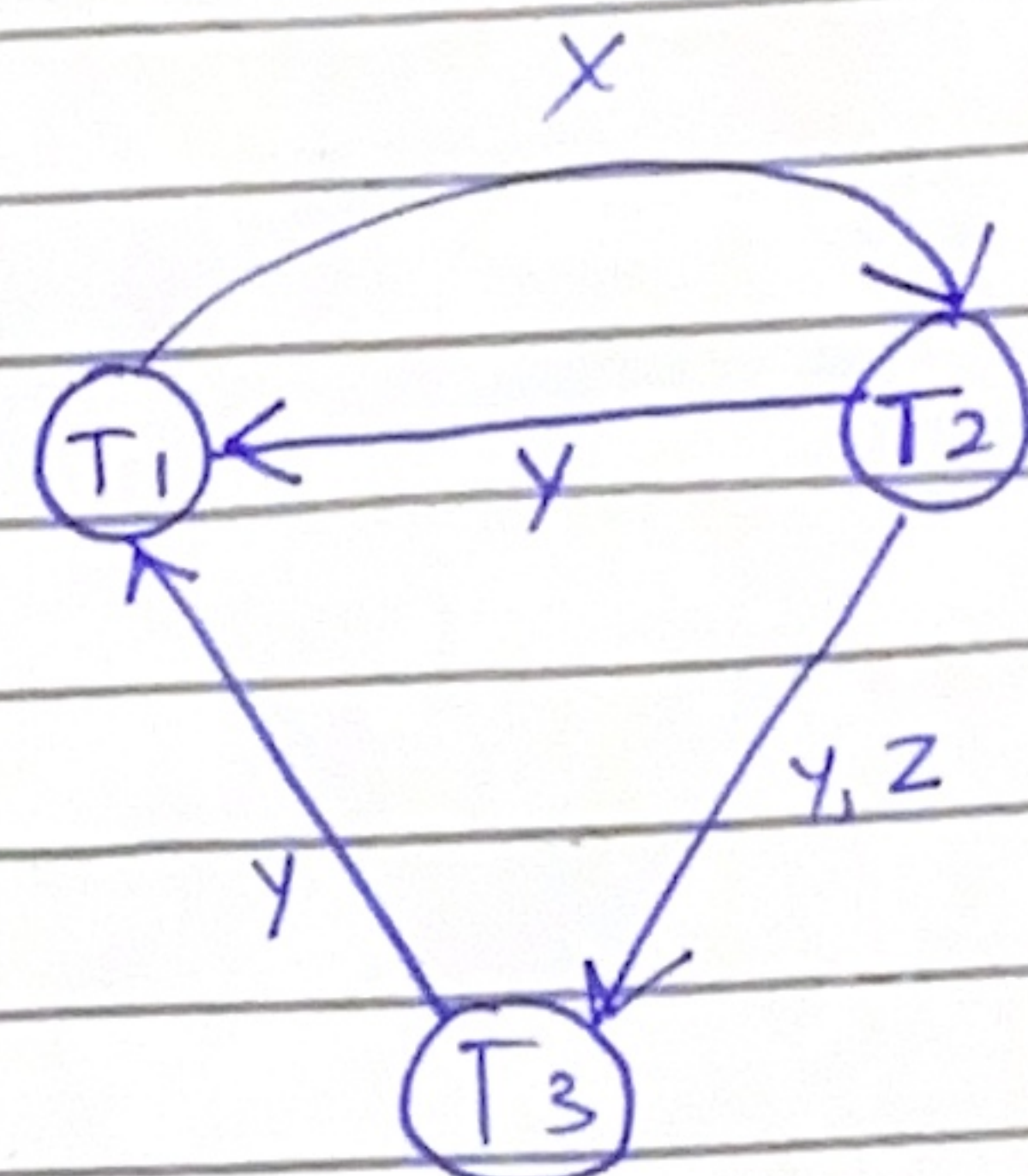
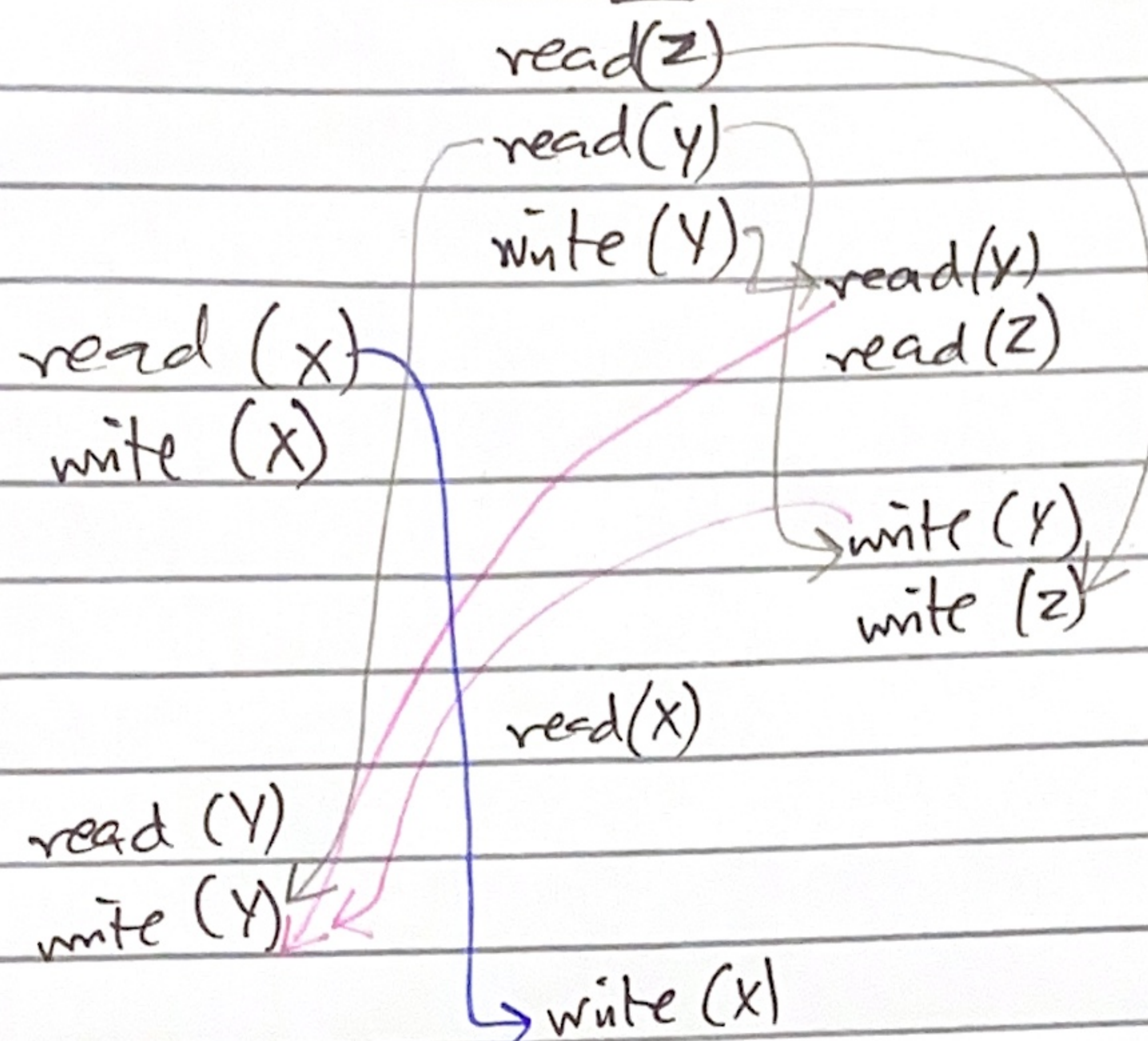
Non-Conflicting Operations.

- $r_1(X) \rightarrow r_2(X)$
- $r_1(X) \rightarrow w_2(Y)$
- $r_1(X) \rightarrow w_1(X)$

~~Graphs~~ Graphs: To identify if schedule is serializable.

- Make ~~the~~ Nodes of each transactions.
- If we've a conflict (means an operation done on same variable in diff. transactions); then we'll make a directed arrow from one node to another. The arrow head will be on the transaction coming back main in that variable.
- If we've a cycle at end; then that schedule isn't serializable. Else w no cycle; we can say if each transaction is runned independently puri tarah first & then second; we'll get the same effect as the question if T_1 & T_2 run parallelly w no cycle wale graph.

Q. T_1 T_2 T_3



★ Since cycle $(T_1 \rightarrow T_2 \rightarrow T_1)$
It is not conflict
serializable

⇒ NOTE:

we Cannot use graphs in practical as DFS
has time consuming problem.
∴ we'll use two phase locks.